EUUI--OSA接入InputSystem后的导航

处理思路

通过持有 EventSystem 引用，将自身设为 selected 对象，由InputSystemUIInputModule 驱动 IMoveHandler / ISubmitHandler / ICancelHandler。

代码

```C#
/// 当前高亮 item index 变化时触发（驱动高亮表现）。支持匿名方法，OnClose 里置 null 清理。
public Action<int> OnItemFocused;

/// 确认选中当前 item 时触发（处理业务数据）。支持匿名方法，OnClose 里置 null 清理。
public Action<int> OnItemSubmitted;

/// 退出列表时触发（将焦点还给面板按钮）。支持匿名方法，OnClose 里置 null 清理。
public Action OnExitList;
```

InputSystemUIInputModule → EventSystem → 暴露的事件

实现的方式是Unity 的**接口派发机制**

手柄/键盘输入
↓
InputSystemUIInputModule（处理 Navigate/Submit/Cancel Action）
↓
EventSystem.Update() 查找 currentSelectedGameObject
↓
对该 GameObject 上的所有 Component 执行接口检查
↓
发现 EUOSAInputBridge 实现了 IMoveHandler / ISubmitHandler / ICancelHandler
↓
调用 OnMove / OnSubmit / OnCancel

所以关键点是：

```C#
public class EUOSAInputBridge : MonoBehaviour, IMoveHandler, ISubmitHandler, ICancelHandler
```

```
_eventSystem.SetSelectedGameObject(gameObject);
```

OnMove：map中的Navigate事件，移动OsaIndex，调用外部的聚焦事件OnItemFocused

C#

```
public void OnMove(AxisEventData eventData)
{
    if (!_isActive) return;

    int delta = 0;
    if (_osa.IsVertical)
    {
        if (eventData.moveDir == MoveDirection.Up)   delta = -1;
        if (eventData.moveDir == MoveDirection.Down) delta =  1;
    }
    else
    {
        if (eventData.moveDir == MoveDirection.Left)  delta = -1;
        if (eventData.moveDir == MoveDirection.Right) delta =  1;
    }

    if (delta == 0) return; // 横向/纵向不属于列表滚动方向，不消费

    int next  = _currentIndex + delta;
    int total = _osa.GetItemsCount();

    if (next < 0 || next >= total)
    {
        if (_loopAtExtremity)
            next = next < 0 ? total - 1 : 0;
        else
        {
            ExitList();
            return;
        }
    }

    _currentIndex = next;
    _osa.BringToView(_currentIndex);
    _osa.BringToView(_currentIndex); // 可变大小 item 需调用两次
    OnItemFocused?.Invoke(_currentIndex);

    eventData.Use();
}
```

OnSubmit：map的submit事件，直接调用外部的OnItemSubmitted，和元素的点击监听同一个即可，手柄不要处理item里面有多个复杂的点击逻辑

```C#
public void OnSubmit(BaseEventData eventData)
{
    if (!_isActive) return;
    OnItemSubmitted?.Invoke(_currentIndex);
}
```

OnCancel：map的cancel事件，会在ExitList触发OnExitList

```C#
public void OnCancel(BaseEventData eventData)
{
    if (!_isActive) return;
    ExitList();
}

/// 退出列表，清除 selected，触发 OnExitList。
public void ExitList()
{
    if (!_isActive) return;
    ForceExit();
    OnExitList?.Invoke();
}
```

导航参数，设置导航的行为

```C#
[Header("导航参数")]
[Tooltip("是否循环滚动到边界")]
[SerializeField] bool _loopAtExtremity  = false;
[Tooltip("退出列表时缓存当前 index，下次 EnterList() 时从该位置恢复而非回到第 0 项")]
[SerializeField] bool _rememberLastIndex = true;
```

两个参数均在进入列表时进行调用

C#

```
///
/// 进入列表。
/// 省略参数时使用 Inspector 配置的默认值；
/// 传入非 null 值时覆盖对应默认值。
///
/// null = 用 Inspector 默认；true/false = 显式控制是否恢复上次位置
/// null = 用 Inspector 默认；true/false = 显式控制到边界时是否循环
public void EnterList(bool? restoreLastIndex = null, bool? loopAtExtremity = null)
{
    EnterListInternal(
        restoreLastIndex ?? _rememberLastIndex,
        loopAtExtremity  ?? _loopAtExtremity
    );
}
```

使用实例

第一次初始化列表的地方如OnShow，传入当前的EventSystem

C#

```
protected override void OnShow()
{
    var loadingList = LoadingList.GetComponent<LoadingListAdapter>();
    this.BindSpriteLoader(loadingList);
    var testData = new List<LoadingListData>();
    for (int i = 0; i < 10; i++)
    {
        testData.Add(new LoadingListData()
        {
            Title = "Test " + i
        });
    }
    LoadingList.gameObject.SetActive(true); // 先激活
    loadingList.SetData(testData);          // 再设数据

    loadingList.InputBridge.SetEventSystem(this.OwnerEventSystem);
}
```

在RegisterEvent中注册列表暴露的回调，以及外部的按钮进入列表EnterList两个参数的设置为导航参数设置

C#

```
private void RegisterUIEvent()
{
    this.AddClick(Start, () =>
    {
        loadingList.InputBridge.EnterList();
    });
    loadingList.InputBridge.OnItemSubmitted = (index) =>
    {
        Debug.Log("Item submitted: " + index);
    };
    loadingList.InputBridge.OnExitList = () =>
    {
        Debug.Log("Exit list");
        //回到面板焦点
        this.Start.Select();
    };
    loadingList.InputBridge.OnItemFocused = (index) =>
    {
        Debug.Log("Item focused: " + index);
    };
}
```

在UnRegisterEvent中，注销回调

C#

```
private void UnRegisterUIEvent()
{
    var loadingList = LoadingList.GetComponent<LoadingListAdapter>();
    loadingList.InputBridge.OnItemSubmitted = null;
    loadingList.InputBridge.OnExitList = null;
    loadingList.InputBridge.OnItemFocused = null;
}
```